State Estimation and Filtering for Robotic Perception

AA273

SLAM, Factor graphs, rigid body poses



Stanford University

MSL

AA273: State Estimation and Filtering for Robotic Perception

Announcements:

- HW 6 due Fri Mar 6
- Project Reports: due Tue Mar 17
  - 5-6 pages + 1 page per group member > 1
  - Latex/Overleaf, IEEE conf format
  - >= 10 refs
  - Describe research and results
  - Re-use proposal material

Lecture 15:

- Factor graphs, SLAM
- Rigid body poses
- Bundle Adjustment
- Pose Graph Optimization

- Project Presentations: 3/10, 3/12
  - 5 min pres + 1 min Q and A
  - Sign up for preferred time slot
  - Share slides through zoom meeting
  - Present on your computer
- Check in with CAs mandatoryin office hours this week

Laplace's Approximation over Full Trajectory

$$p(x_{0:T} \mid y_{1:T}) \approx \mathcal{N}(x^*_{0:T}, \Sigma^*_{0:T})$$

Where:

$$x^*_{0:T} = \arg\min_{x_{0:T}} \sum_{t=1}^{T} \left(\frac{1}{2} \|y_t - g(x_t)\|^2_{R_t^{-1}} + \frac{1}{2} \|x_t - f(x_{t-1}, u_{t-1})\|^2_{Q_t^{-1}} \right) + \frac{1}{2} \|x_0 - \mu_0\|^2_{\Sigma_0^{-1}}$$

$$\Sigma^*_{0:T} = \nabla^2_{x_{0:T}} \left[\sum_{t=1}^{T} \left(\frac{1}{2} \|y_t - g(x_t)\|^2_{R_t^{-1}} + \frac{1}{2} \|x_t - f(x_{t-1}, u_{t-1})\|^2_{Q_t^{-1}} \right) + \frac{1}{2} \|x_0 - \mu_0\|^2_{\Sigma_0^{-1}} \right]$$

Can solve with Gauss-Newton or OTS nonlinear optimization solver (e.g., Gurobi)

$x^*_{0:T}$ Called **smoothed** trajectory estimate

x^*_t Similar to EKF mean, but with **smaller covariance** Σ^*_t

Uses **measurements in the future** $y_\tau, \tau > t$, to improve state estimate at x_t

Factor Graph Interpretation

$$-\log(p(x_{0:T} \mid y_{1:T})) = \sum_{t=1}^{T} \left(\frac{1}{2}\|y_t - g(x_t)\|^2_{R_t^{-1}} + \frac{1}{2}\|x_t - f(x_{t-1}, u_{t-1})\|^2_{Q_t^{-1}} \right) + \frac{1}{2}\|x_0 - \mu_0\|^2_{\Sigma_0^{-1}}$$

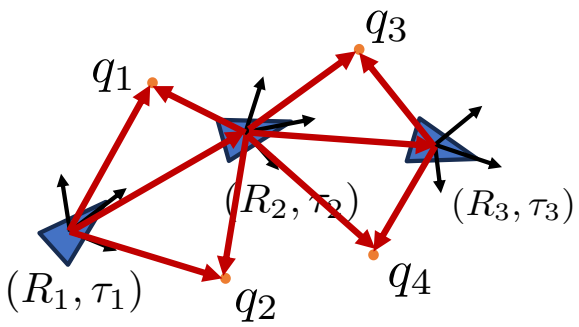
Measurement factor

Dynamics factor

Prior factor

See details on the board and in the notes.

Factor Graph



(R_i, τ_i)
q_k ○ Unknown quantities

f_{ik} □ "Factors": Costs associated with measurements

Solve for

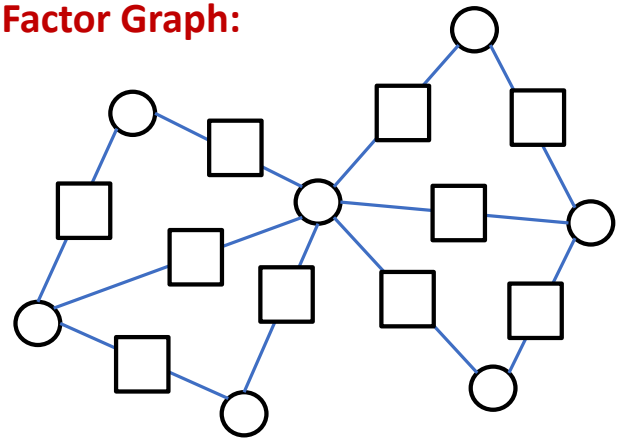
Robot trajectory: $((R_1, \tau_1), \ldots, (R_T, \tau_T))$

Map: $(q_1, \ldots, q_M)$

Given: Sensor Measurements

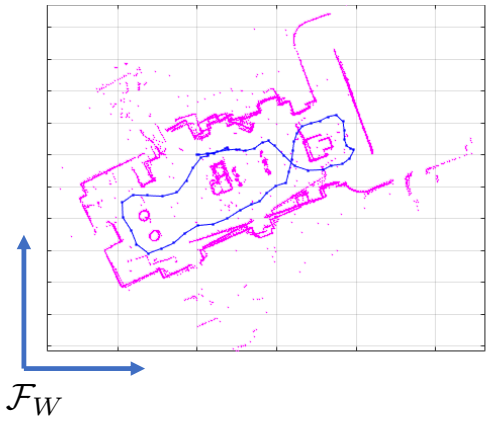
- RGB, Point Cloud, RGB-D
- IMU/Wheel encoder odometry

Factor Graph:



Point Cloud Map

2D:



Points represent occupied space:

$$q_i \in \mathbb{R}^2 \text{ or } \mathbb{R}^3$$

Native output of a lidar sensor.

$$Q_{\text{occ}} = \{q_i\}_{i=1}^{N}$$

Still need to transform from robot body to world frame.

Probabilistic point cloud map:

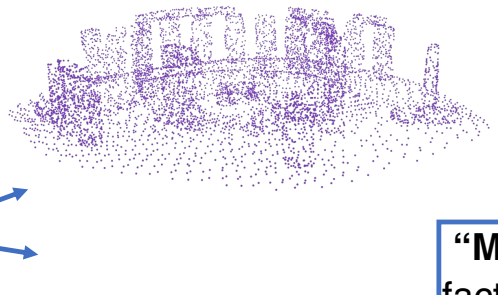
Multivariate Gaussian

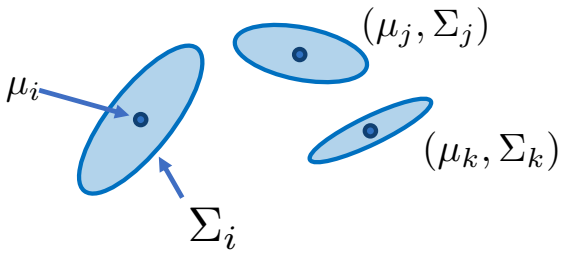
$$p(q_i \mid \text{robot measurements}) = \mathcal{N}(\mu_i, \Sigma_i)$$

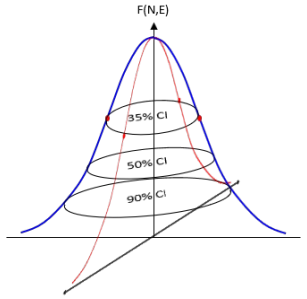
Mean vector

Covariance matrix

3D:







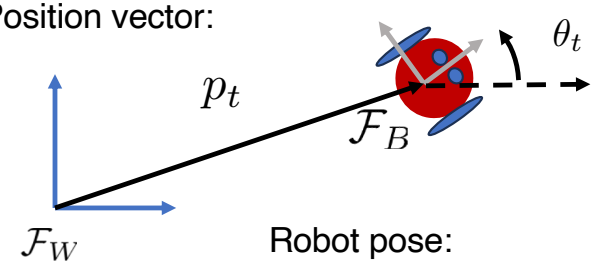
"Mapping": Update (μ_i, Σ_i) with a Kalman filter or a factor graph optimization solver.

Coordinate Frames and Transforms

Robot body in body-fixed frame: $B \subset \mathbb{R}^2$ or $\mathbb{R}^3$

Position vector:

Heading angle:



Robot pose:

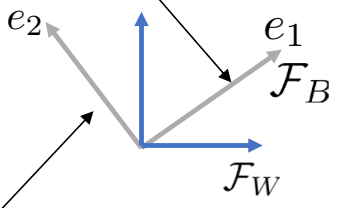
(p_t, θ_t) or (p_t, R_t)

Rotation matrix:

$$R_t = \begin{bmatrix} \cos\theta_t & -\sin\theta_t \\ \sin\theta_t & \cos\theta_t \end{bmatrix}$$

$e_1 = \begin{bmatrix} \cos\theta \\ \sin\theta \end{bmatrix}$ e_1 expressed in $\mathcal{F}_W$

$e_2 = \begin{bmatrix} -\sin\theta \\ \cos\theta \end{bmatrix}$ e_2 expressed in $\mathcal{F}_W$



Transform vector from body to world:

$$v_W = R_t v_B + p_t \quad \text{or} \quad \begin{bmatrix} v_W \\ 1 \end{bmatrix} = \begin{bmatrix} R_t & p_t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v_B \\ 1 \end{bmatrix}$$

Robot body in world frame:

$$B(p_t, R_t) = \{R_t b + p_t \mid b \in B\}$$

Recall, special orthogonal matrix:

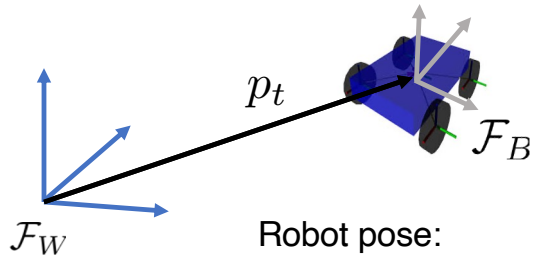
$$R_t \in \mathbb{SO}(2) \text{ or } \mathbb{SO}(3)$$

Definition: $R_t^T R_t = I$ and $\det(R_t) = +1$

All columns mutually orthogonal unit vectors.

More detail on 3D

Robot body in body-fixed frame: $B \subset \mathbb{R}^2$ or $\mathbb{R}^3$



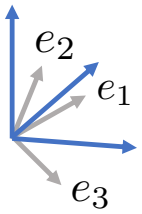
Robot pose:

(p_t, R_t)

Rotation matrix:

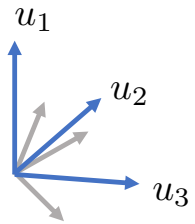
$$R_t = \begin{bmatrix} e_1 & e_2 & e_3 \end{bmatrix}$$

e_i expressed in $\mathcal{F}_W$



Inverse transform:

$$R_t = \begin{bmatrix} u_1^T \\ u_2^T \\ u_3^T \end{bmatrix}$$



$$R_t^T = R_t^{-1} = \begin{bmatrix} u_1 & u_2 & u_3 \end{bmatrix}$$

Transform vector from body to world:

$$v_W = R_t v_B + p_t \quad \text{or} \quad \begin{bmatrix} v_W \\ 1 \end{bmatrix} = \begin{bmatrix} R_t & p_t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v_B \\ 1 \end{bmatrix}$$

Robot body in world frame:

$$B(p_t, R_t) = \{R_t b + p_t \mid b \in B\}$$

Recall, special orthogonal matrix:

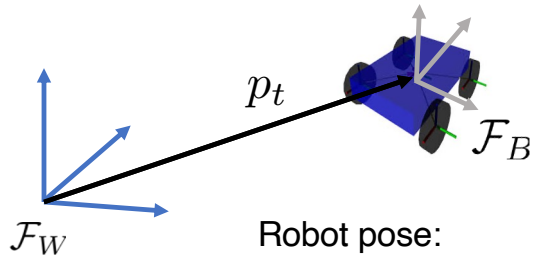
$$R_t \in \mathbb{SO}(2) \text{ or } \mathbb{SO}(3)$$

Definition: $R_t^T R_t = I$ and $\det(R_t) = +1$

All columns mutually orthogonal unit vectors.

Rigid Body Transforms

Robot body in body-fixed frame: $B \subset \mathbb{R}^2$ or $\mathbb{R}^3$



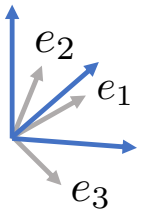
Robot pose:

(p_t, R_t)

Rotation matrix:

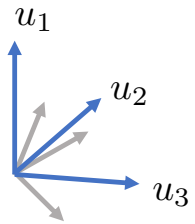
$$R_t = \begin{bmatrix} e_1 & e_2 & e_3 \end{bmatrix}$$

e_i expressed in $\mathcal{F}_W$



Inverse transform:

$$R_t = \begin{bmatrix} u_1^T \\ u_2^T \\ u_3^T \end{bmatrix}$$



$$R_t^T = R_t^{-1} = \begin{bmatrix} u_1 & u_2 & u_3 \end{bmatrix}$$

Transform vector from body to world:

$$v_W = R_t v_B + p_t \quad \text{or} \quad \begin{bmatrix} v_W \\ 1 \end{bmatrix} = \overbrace{\begin{bmatrix} R_t & p_t \\ 0 & 1 \end{bmatrix}}^{T_t} \begin{bmatrix} v_B \\ 1 \end{bmatrix}$$

Inverse transform:

$$v_B = R_t^T v_W - R_t^T p_t \quad \text{or} \quad \begin{bmatrix} v_B \\ 1 \end{bmatrix} = \overbrace{\begin{bmatrix} R_t^T & -R_t^T p_t \\ 0 & 1 \end{bmatrix}}^{T_t^{-1}} \begin{bmatrix} v_W \\ 1 \end{bmatrix}$$

Special Euclidean group (rigid body transforms):

$$(p_t, R_t) \in \mathbb{SE}(2) \text{ or } \mathbb{SE}(3)$$

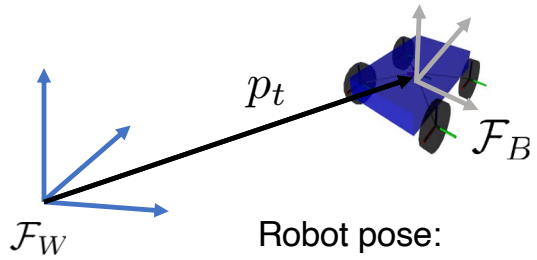
and $(-R_t^T p_t, R_t^T) \in \mathbb{SE}(2)$ or $\mathbb{SE}(3)$

we can also say: $T_t \in \mathbb{SE}(2)$ or $\mathbb{SE}(3)$

and $T_t^{-1} \in \mathbb{SE}(2)$ or $\mathbb{SE}(3)$

Rotation Matrix Parameterizations

Robot body in body-fixed frame: $B \subset \mathbb{R}^2$ or $\mathbb{R}^3$



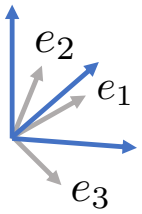
Robot pose:

(p_t, R_t)

Rotation matrix:

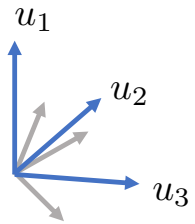
$$R_t = \begin{bmatrix} e_1 & e_2 & e_3 \end{bmatrix}$$

e_i expressed in $\mathcal{F}_W$



Inverse transform:

$$R_t = \begin{bmatrix} u_1^T \\ u_2^T \\ u_3^T \end{bmatrix}$$



Parameterizations:

- Rotation matrix R_t
- Euler angles (ϕ, ψ, θ)
- Quaternions (q_1, q_2, q_3, q_4)
- Angle-axis $(\omega_1, \omega_2, \omega_3)$

All have 3 degrees of freedom.

Must have at least 4 parameters)

to represent $\mathbb{SO}(3)$ without

singularities.

Rotation matrix:

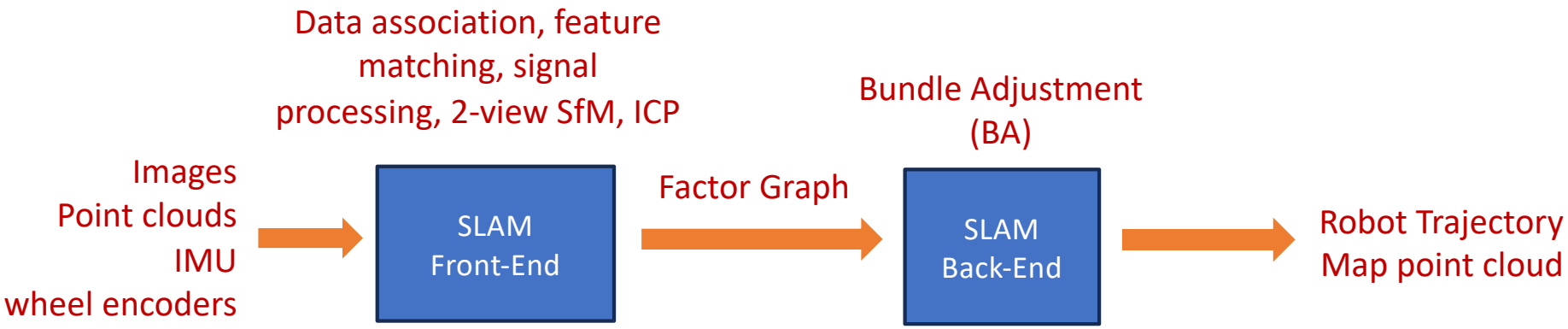
R_t 9 parameters

$R_t^T R_t = I$ 6 constraints

= 3 degrees of freedom

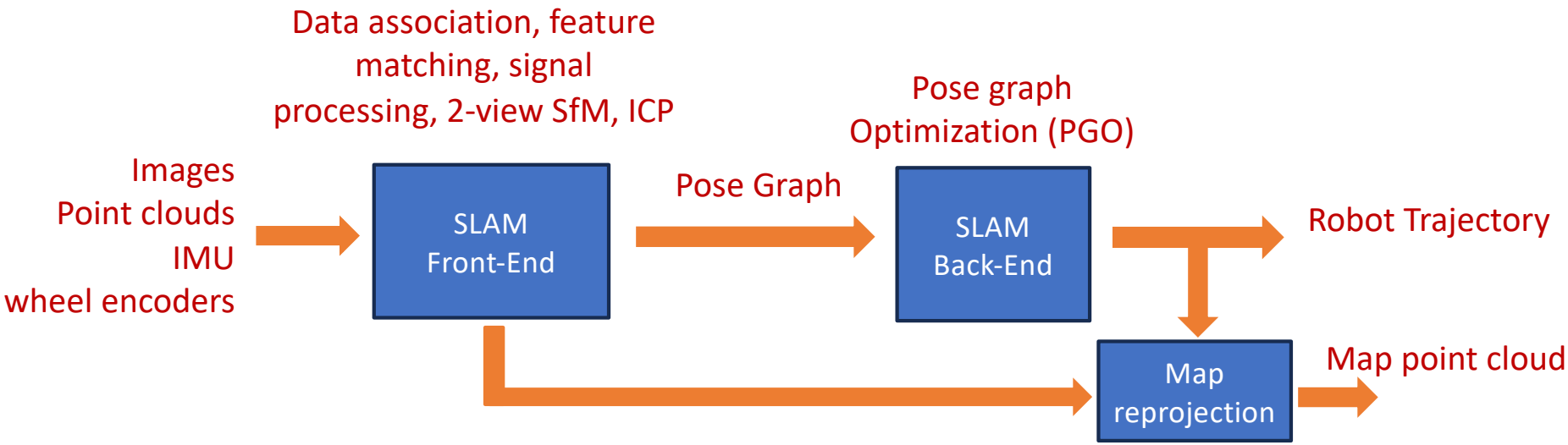
Factor Graphs: A Unifying Perspective for SLAM

(Frank Dellaert, 2012, 2017)



Bundle Adjustment SLAM - Produces both robot traj and map point cloud

Factor Graphs: A Unifying Perspective for SLAM



Pose Graph SLAM - Produces robot traj first, map obtained in post processing.

NeRFBridge for SLAM with NeRFs

NerfBridge: Bringing Real-time, Online Neural Radiance Field Training to Robotics

Javier Yu, Jun En Low, Keiko Nagami, and Mac Schwager

Stanford University

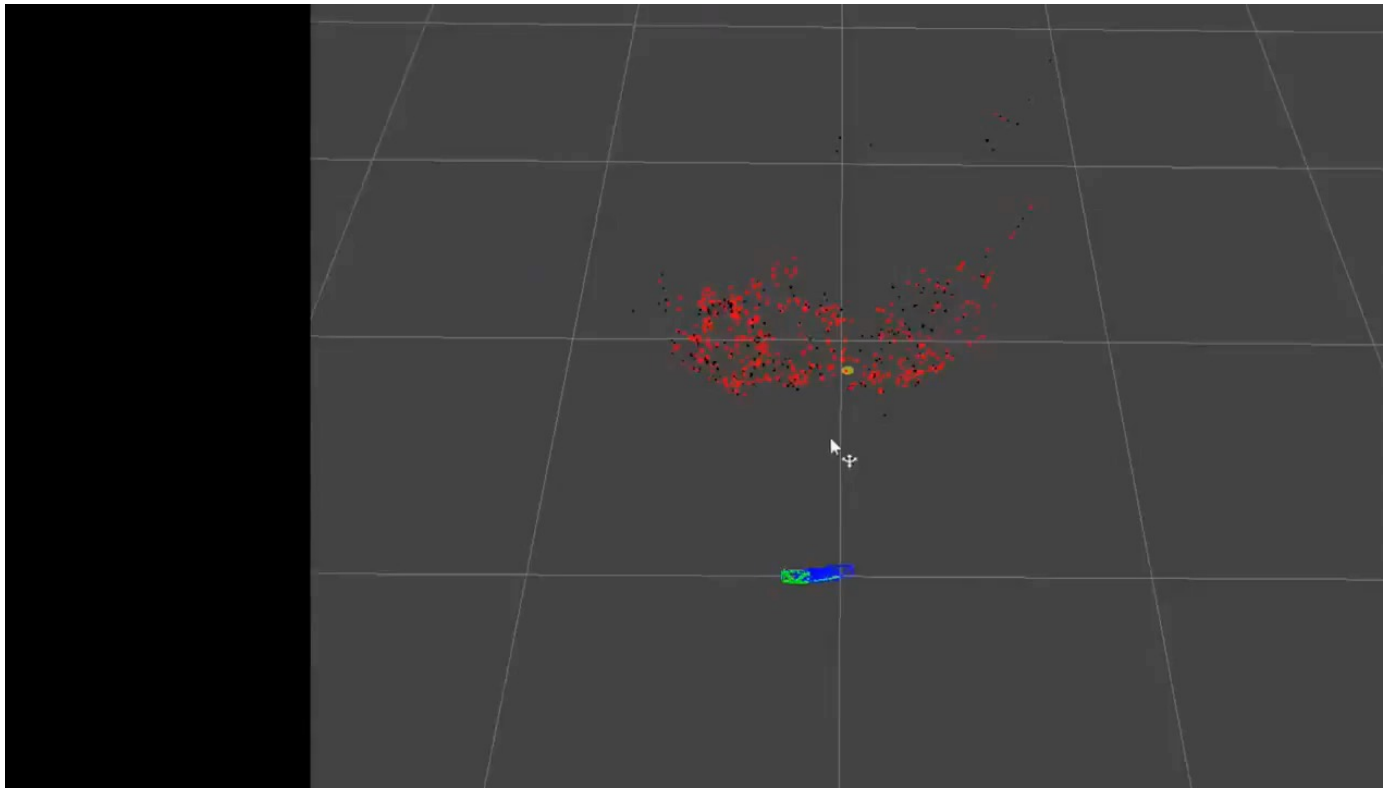
Contact: javieryu@stanford.edu
Code: https://github.com/javieryu/nerf_bridge



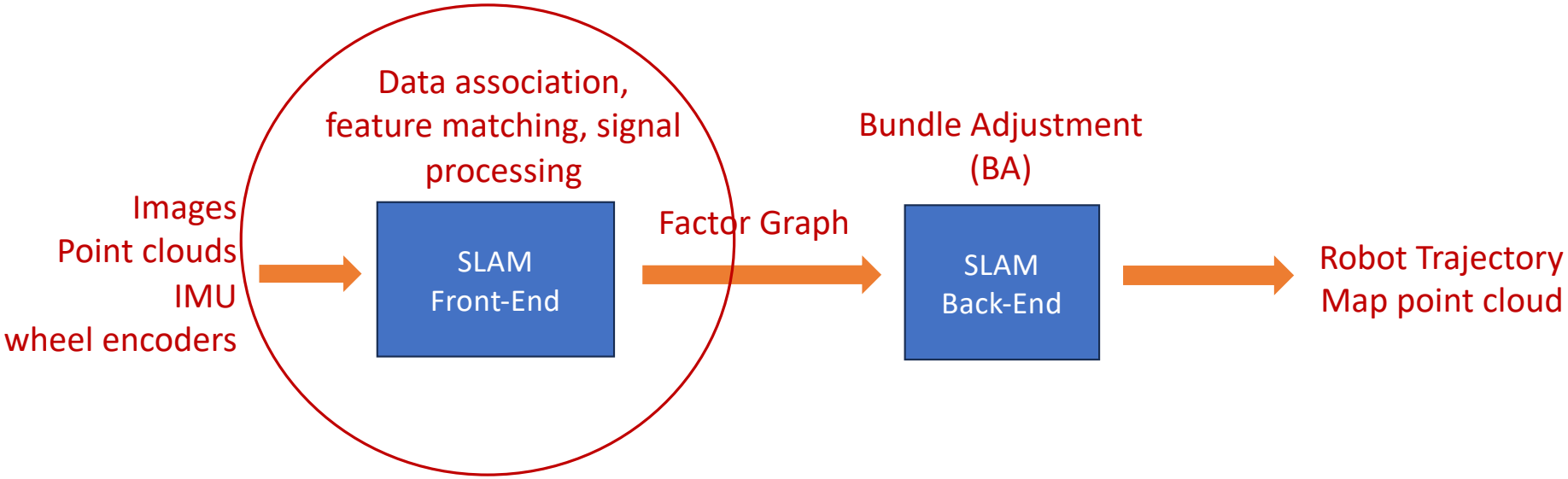
SLAM in ROS NAV2 toolbox



SLAM in Space for Asteroid Reconstruction

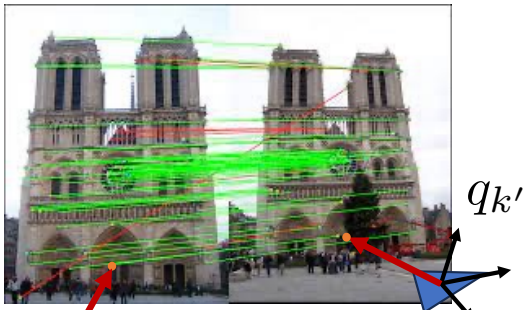


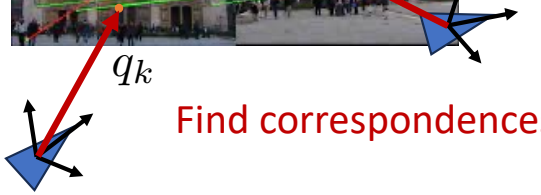
Factor Graphs: A Unifying Perspective for Modern SLAM



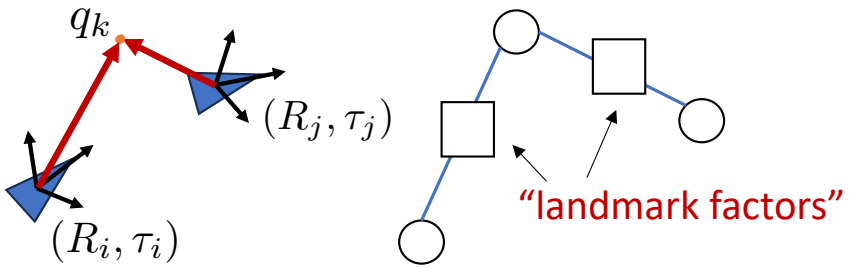
Front End: Building the factor graph

For RGB, RGB-D sensors:

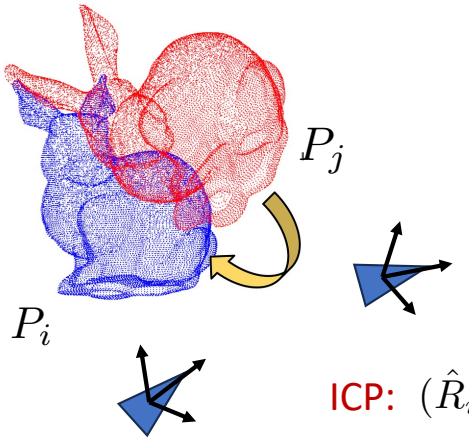




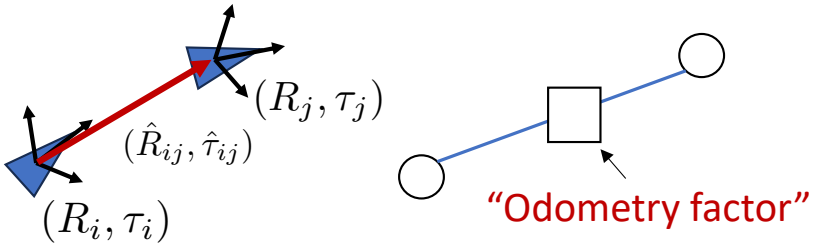
Find correspondences: $q_k = q_{k'}$



For Lidar:

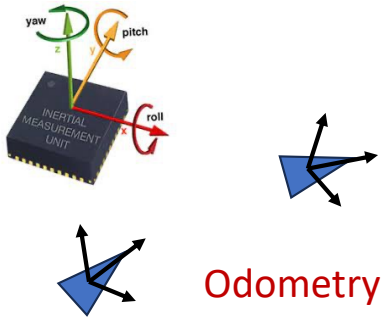


ICP: $(\hat{R}_{ij}, \hat{\tau}_{ij})$



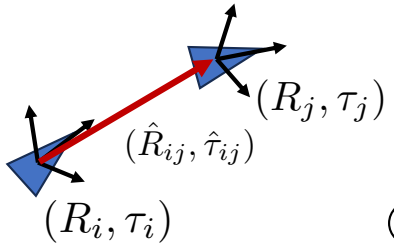
Front End: Building the factor graph

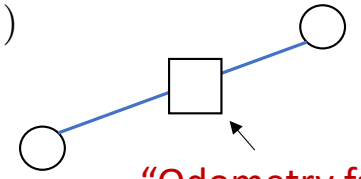
For IMU/encoder odometry:



Odometry (usually with EKF):

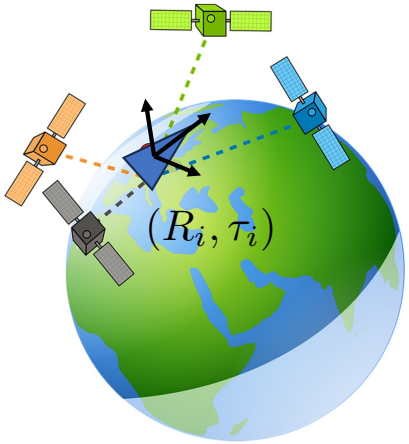
$(\hat{R}_{ij}, \hat{\tau}_{ij})$



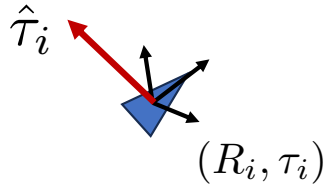


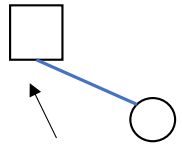
"Odometry factor"

GPS:



GPS: $\hat{\tau}_i$

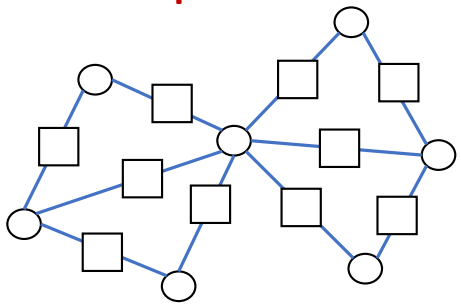


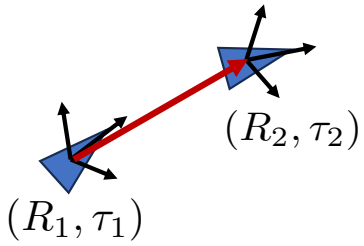


"GPS factor"
"Strapdown factor"

Factors: Lidar and Odometry measurements

Factor Graph:



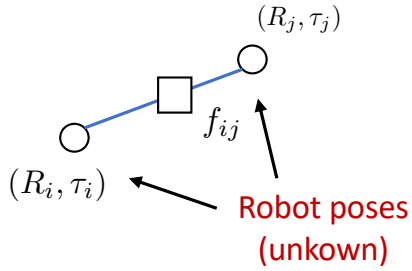


Odometry measurement: $(\hat{R}_{ij}, \hat{\tau}_{ij})$

$$\hat{R}_{ij} = R_i^T R_j + v_{ij}^R$$
$$\nu_{ij}^R \sim \mathcal{N}(0, \Sigma_{ij}^R)$$

$$\hat{\tau}_{ij} = R_i^T (\tau_j - \tau_i) + \nu_{ij}^\tau$$
$$\nu_{ij}^\tau \sim \mathcal{N}(0, \Sigma_{ij}^\tau)$$

Odometry factor



Odometry factor:

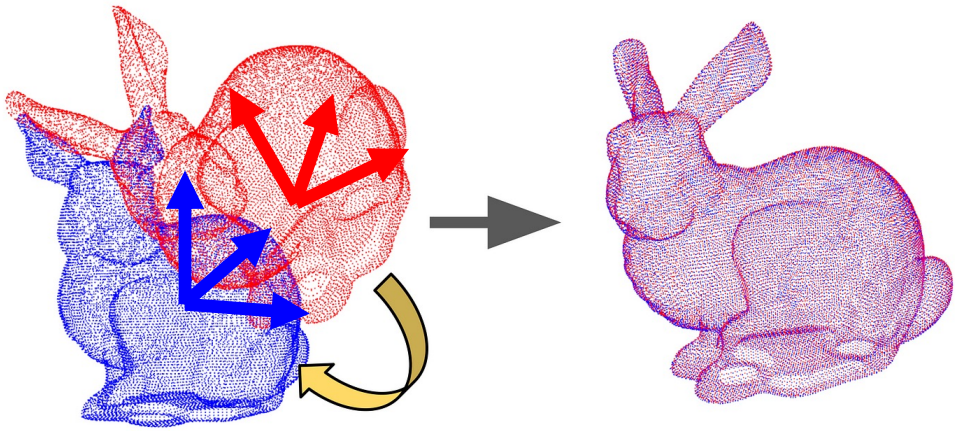
$$f_{ij} = \lambda_{ij}^R \| R_i \hat{R}_{ij} - R_j \|^2 + \lambda_{ij}^\tau \| R_i \hat{\tau}_{ij} - (\tau_j - \tau_i) \|^2$$

$$\lambda_{ij}^R = \det(\Sigma_{ij}^R)^{-1} \qquad \lambda_{ij}^\tau = \det(\Sigma_{ij}^\tau)^{-1}$$

Weighting constants

Interpretation: Cost of *inconsistency* with measurement.

Point cloud alignment: Iterative Closest Point (ICP)



- Obtain the relative pose: The pose transform required to move P_B to align with P_A

$$(\tau_{AB}, R_{AB})$$

Translation vector

Rotation matrix

Basics and notation:

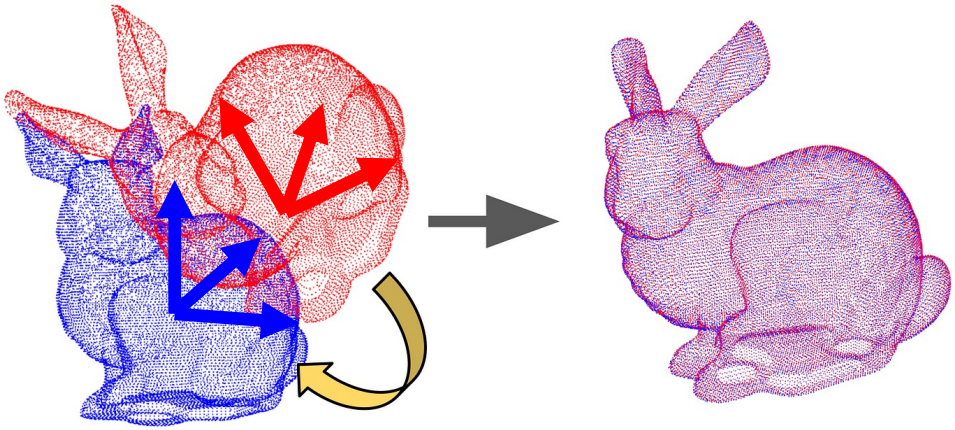
- Rigid body transforms, SE(2), SE(3)

$$(\tau_{AB}, R_{AB}) \in \mathrm{SE}(2) \text{ or } \mathrm{SE}(3)$$

$$P_A = R_{AB} P_B + \tau_{AB}$$

rotate

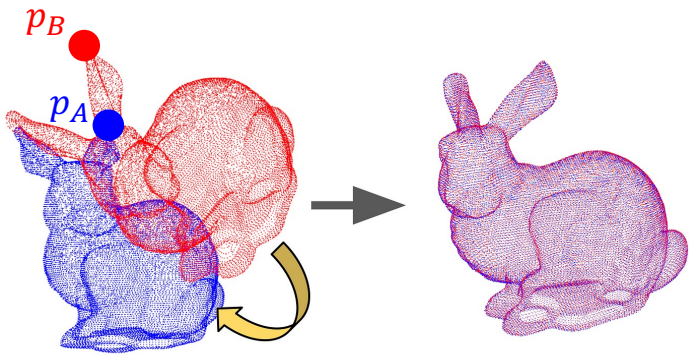
translate



Alignment with Known Correspondences

- Find (τ_{AB}, R_{AB})

$$P_A = R_{AB}P_B + \tau_{AB}$$



$$\min_{(\tau_{AB}, R_{AB})} \|P_A - (R_{AB}P_B + \tau_{AB})\|_{\mathcal{F}}^2$$

$$\text{s.t.} \quad R_{AB}^T R_{AB} = I$$

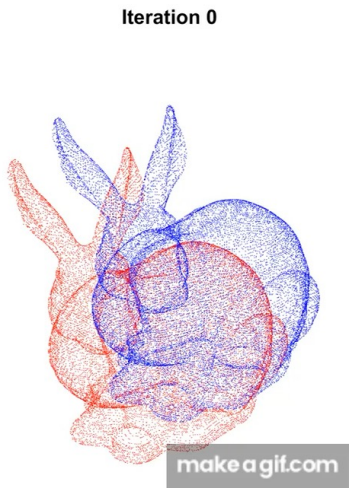
- Known solution based on least squares - Singular Value Decomposition (SVD)

Iterative Closest Point (ICP): iterate with closest point heuristic

- For each point in P_B

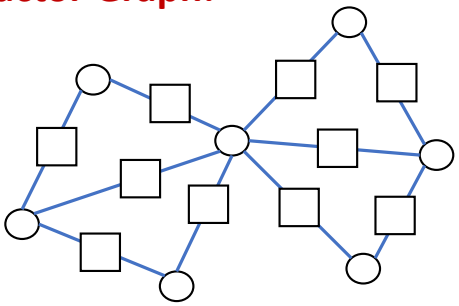
  p_i^B match to closest point p_j^A from P_A

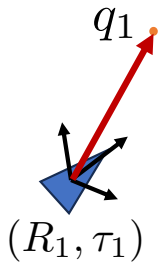
- Solve for (τ_{AB}, R_{AB}) with least squares
- Repeat until convergence



Factors: RGB-D measurements

Factor Graph:





RGB-D measurement:

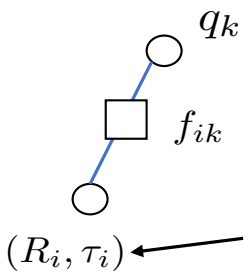
$$\hat{q}_{ik} = R_i^T(q_k - \tau_i) + \nu_{ik}^D$$

3D feature location in robot's frame (measured)

noise

$$\nu_{ik}^D \sim \mathcal{N}(0, \Sigma_{ik}^D)$$

RGB-D factor



3D feature location in global frame (unknown)

Robot pose (unkown)

RGB-D factor:

$$f_{ik} = \lambda_{ik}^D \| R_i \hat{q}_{ik} - (q_k - \tau_i) \|^2$$

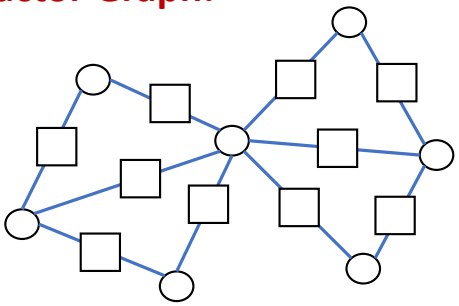
Weighting constant

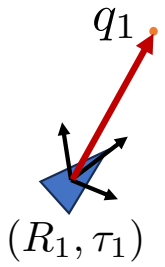
$$\lambda_{ik}^D = \det(\Sigma_{ik}^D)^{-1}$$

Interpretation: Cost of *inconsistency* with measurement.

Factors: RGB measurements

Factor Graph:





Camera calibration matrix

noise $\nu_{ik}^V \sim \mathcal{N}(0, \Sigma_{ik}^V)$

RGB measurement:

$$p_{ik}^h = K R_i^T (q_k - \tau_i) + \nu_{ik}$$

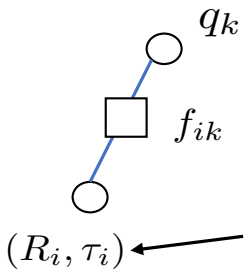
Homogeneous pixel coords

$$p_{ik}^h = (u_{ik}, v_{ik}, 1) z_{ik}^h$$

Measured pixel coords in image

Unknown pixel depth

RGB-D factor



3D feature location in global frame (unknown)

Robot pose (unkown)

RGB factor:

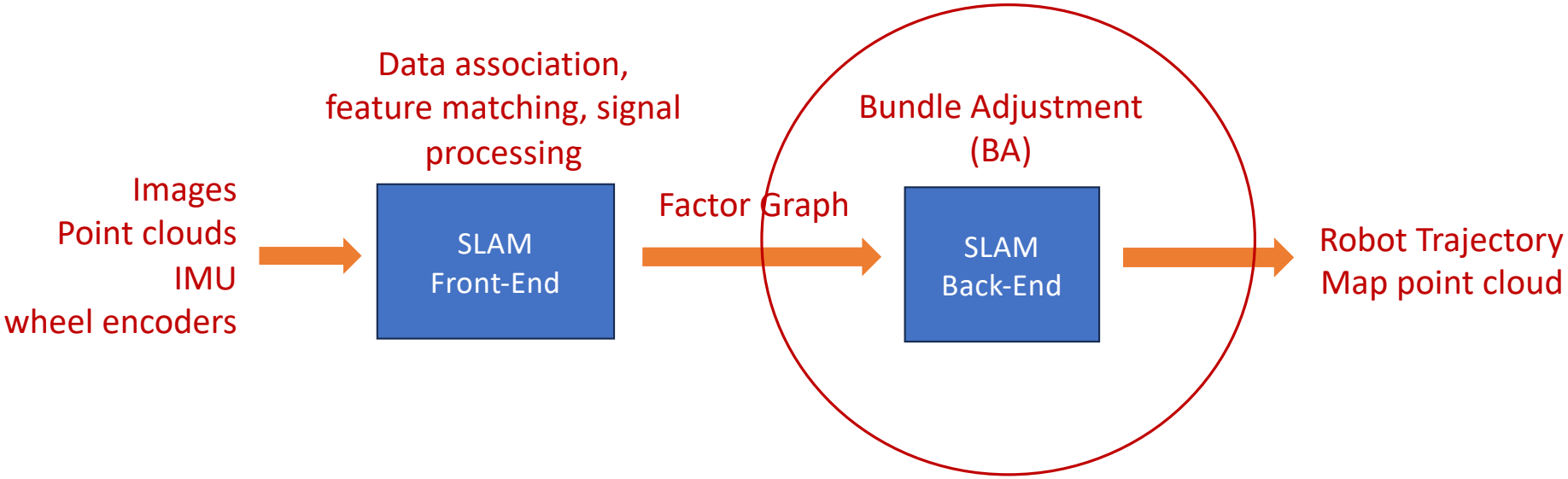
$$f_{ik} = \lambda_{ik}^V \| R_i K^{-1} p_{ik}^h - (q_k - \tau_i) \|^2$$

Weighting constant

$$\lambda_{ik}^V = \det(\Sigma_{ik}^V)^{-1}$$

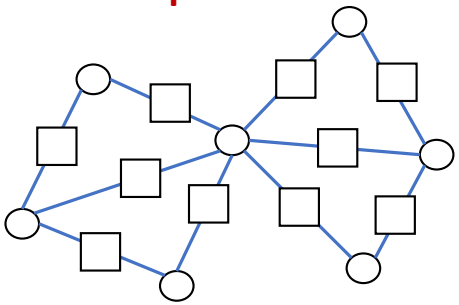
Interpretation: Cost of *inconsistency* with measurement.

Factor Graphs: Bundle Adjustment



Bundle Adjustment Optimization

Factor Graph:



Sum of all factors gives total cost:

$$J((R_i,\tau_i),q_k) = \sum_{(\hat{R}_{ij},\hat{\tau}_{ij})} (\lambda_{ij}^R \|R_i\hat{R}_{ij} - R_j\|^2 + \lambda_{ij}^\tau \|R_i\hat{\tau}_{ij} - (\tau_j - \tau_i)\|^2)$$
$$+ \sum_{\hat{q}_{ik}} \lambda_{ik}^D \|R_i\hat{q}_{ik} - (q_k - \tau_i)\|^2 + \sum_{(u_{ik},v_{ik})} \lambda_{ik}^V \|R_i K^{-1} p_{ik}^h - (q_k - \tau_i)\|^2$$

Optimization Problem (Bundle Adjustment) :

$$\min_{(R_i,\tau_i),q_k} J((R_i,\tau_i),q_k)$$
$$\text{subject to} \quad \left.\begin{array}{r} R_i^T R_i = I \\ \det(R_i) = 1 \end{array}\right\} \; SO(3)$$

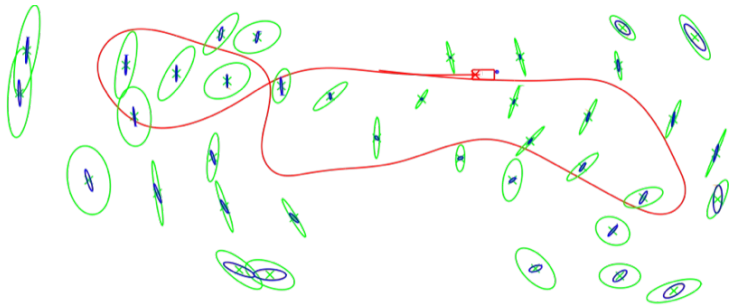
Interpretation: Find poses and feature locations that are *as consistent as possible* with all the measurements.

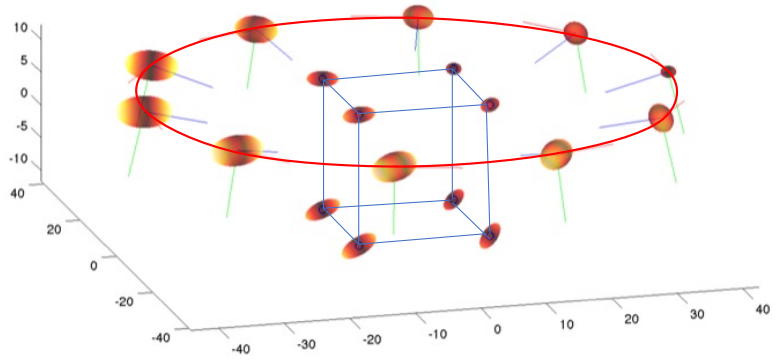
SLAM pose and map estimates:

$$\{(R_i^*,\tau_i^*),q_k^*\} = \arg\min_{(R_i,\tau_i),q_k} J((R_i,\tau_i),q_k)$$

GTSAM: The Standard Open-Source Factor Graph Optimizer

Frank Dellaert, and collaborators, GA Tech

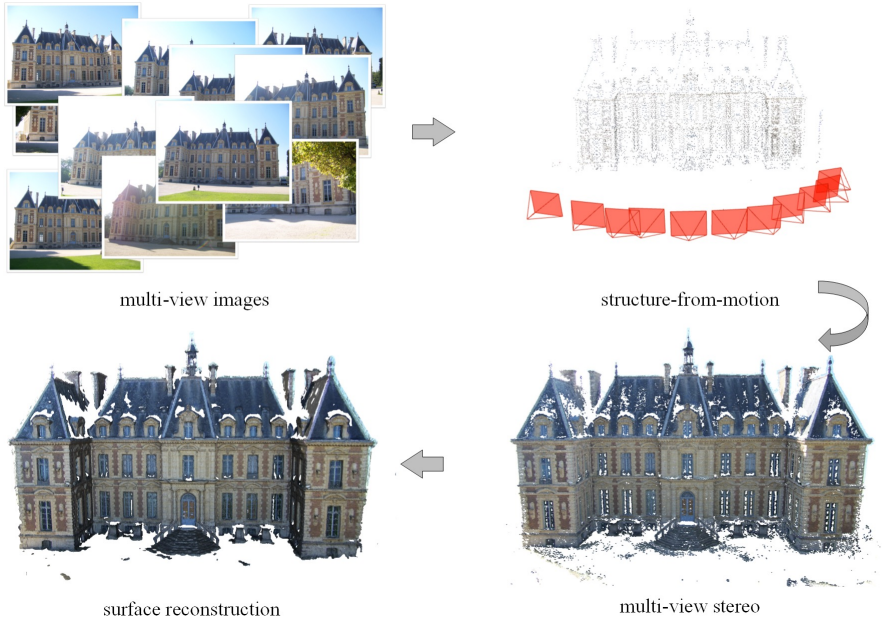




https://gtsam.org/

Colmap: The Standard Open-Source CV 3D Photogrammetry Package

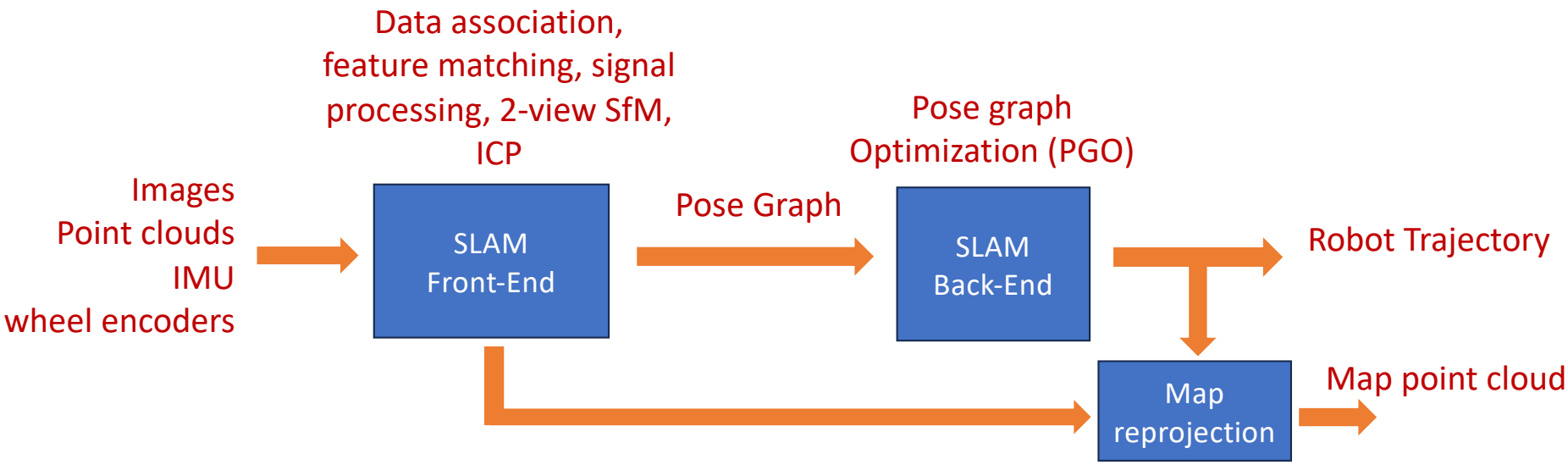
Johannes Schönberger, and collaborators, ETH Zurich





https://github.com/colmap/colmap

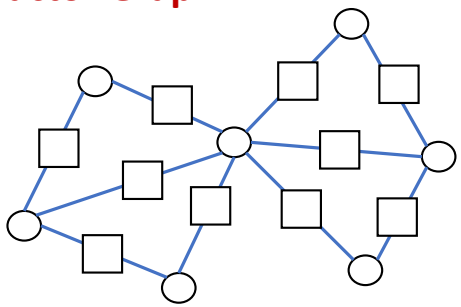
Pose Graph SLAM



Pose Graph SLAM - Produces robot traj first, map obtained in post processing.

Bundle Adjustment Optimization

Factor Graph:



Sum of all factors gives total cost:

Odometry factors (LIDAR, IMU/encoders)

$$J((R_i,\tau_i),q_k) = \sum_{(\hat{R}_{ij},\hat{\tau}_{ij})} (\lambda_{ij}^R \|R_i\hat{R}_{ij} - R_j\|^2 + \lambda_{ij}^\tau \|R_i\hat{\tau}_{ij} - (\tau_j - \tau_i)\|^2)$$

$$+ \sum_{\hat{q}_{ik}} \underbrace{\lambda_{ik}^D \|R_i\hat{q}_{ik} - (q_k - \tau_i)\|^2}_{\text{RGB-D factors}} + \sum_{(u_{ik},v_{ik})} \underbrace{\lambda_{ik}^V \|R_i K^{-1} p_{ik}^h - (q_k - \tau_i)\|^2}_{\text{RGB factors}}$$

Optimization Problem (Bundle Adjustment) :

$$\min_{(R_i,\tau_i),q_k} J((R_i,\tau_i),q_k)$$

$$\text{subject to} \quad \left.\begin{aligned} R_i^T R_i = I \\ \det(R_i) = 1 \end{aligned}\right\} \; SO(3)$$

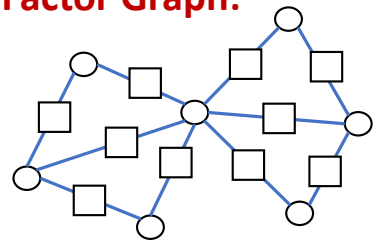
Interpretation: Find poses and feature locations that are *as consistent as possible* with all the measurements.

SLAM pose and map estimates:

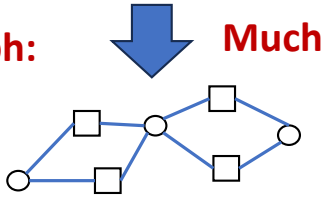
$$\{(R_i^*,\tau_i^*),q_k^*\} = \arg\min_{(R_i,\tau_i),q_k} J((R_i,\tau_i),q_k)$$

Pose Graph Optimization

Factor Graph:



Pose Graph:



Much simpler!

Interpretation: Still looking for poses *most consistent* with all relative pose measurements. Typically **much faster** than full bundle adjustment, but can be **less accurate**.

Pose Graph Cost Function:

Odometry factors (from all sensors)

$$J((R_i,\tau_i)_{i=1}^N) = \sum_{(\hat{R}_{ij},\hat{\tau}_{ij})} (\lambda_{ij}^R \|R_i\hat{R}_{ij} - R_j\|^2 + \lambda_{ij}^\tau \|R_i\hat{\tau}_{ij} - (\tau_j - \tau_i)\|^2)$$

Pose Graph Optimization (PGO):

$$\min_{(R_i,\tau_i)_{i=1}^N} J((R_i,\tau_i)_{i=1}^N)$$

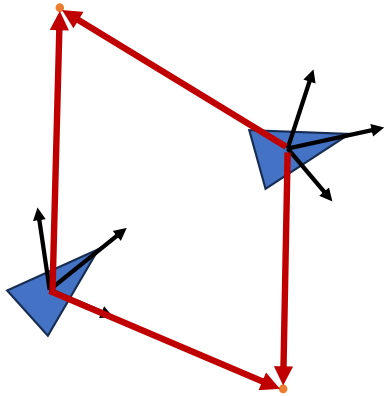
$$\text{subject to} \quad R_i^T R_i = I$$

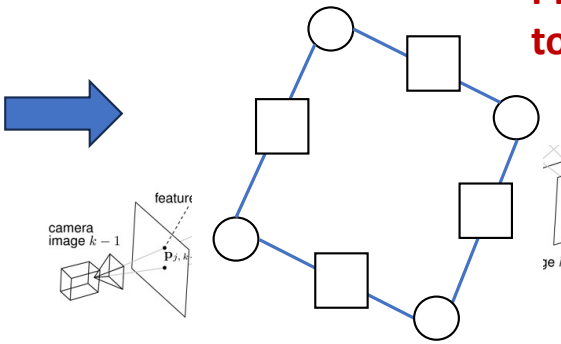
$$\det(R_i) = 1$$

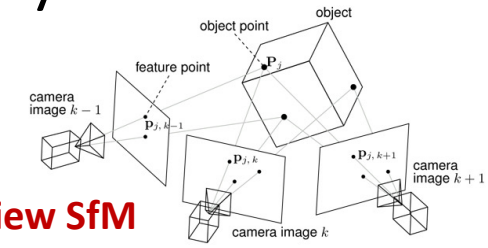
SLAM pose estimates only (no map!):

$$\{(R_i^*,\tau_i^*)_{i=1}^N\} = \arg\min_{(R_i,\tau_i)_{i=1}^N} J((R_i,\tau_i)_{i=1}^N)$$

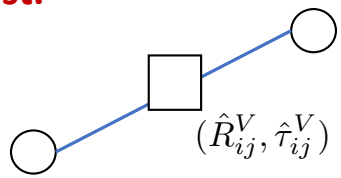
Pose Graph: A Factor Graph with Only Odometry Factors!



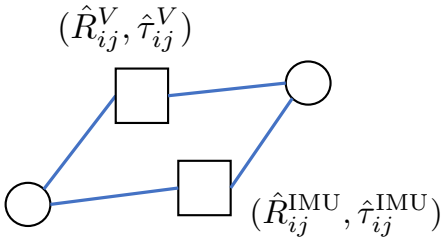




Pre-process with 2 view SfM to get relative pose est.



e.g. with lidar and IMU/wheel odometry



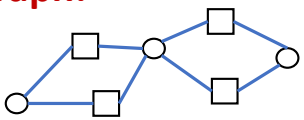
Interpretation: Do more processing in front end to get a simpler optimization in the back end.

Obtaining the Map from PGO Solution

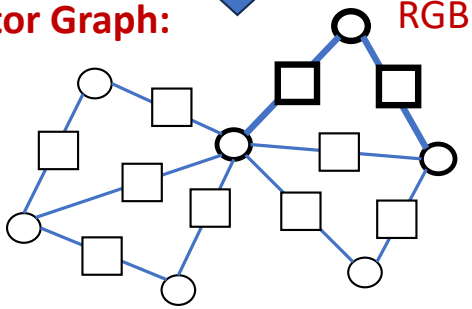
SLAM pose estimates:

$$\{(R_i^*, \tau_i^*)_{i=1}^N\} = \arg \min_{(R_i,\tau_i)_{i=1}^N} J((R_i, \tau_i)_{i=1}^N)$$

Pose Graph:

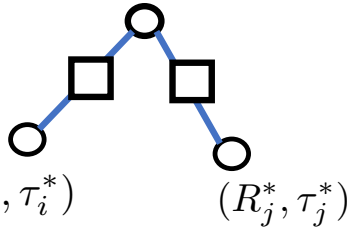


Factor Graph: RGB factors



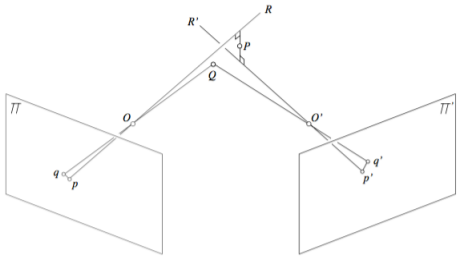
Point in point cloud

q_k^* unknown



known

Stereo Depth: find location of point in 3D from two RGB images with known poses.

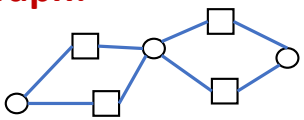


Obtaining the Map from PGO solution

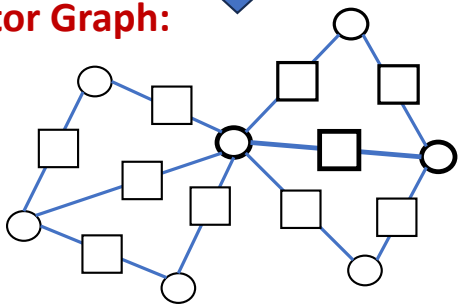
SLAM pose estimates:

$$\{(R_i^*, \tau_i^*)_{i=1}^N\} = \arg \min_{(R_i,\tau_i)_{i=1}^N} J((R_i, \tau_i)_{i=1}^N)$$

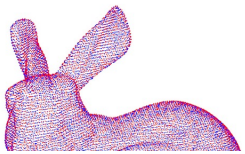
Pose Graph:

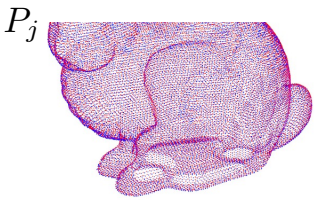


Factor Graph:



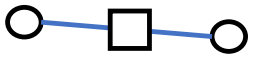
Lidar point clouds in local frames





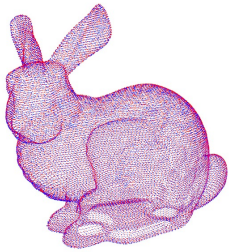
$P_i^* = R_i^* P_i + \tau_i^*$

$P_j^* = R_j^* P_j + \tau_j^*$



(R_i^*, τ_i^*) (R_j^*, τ_j^*)

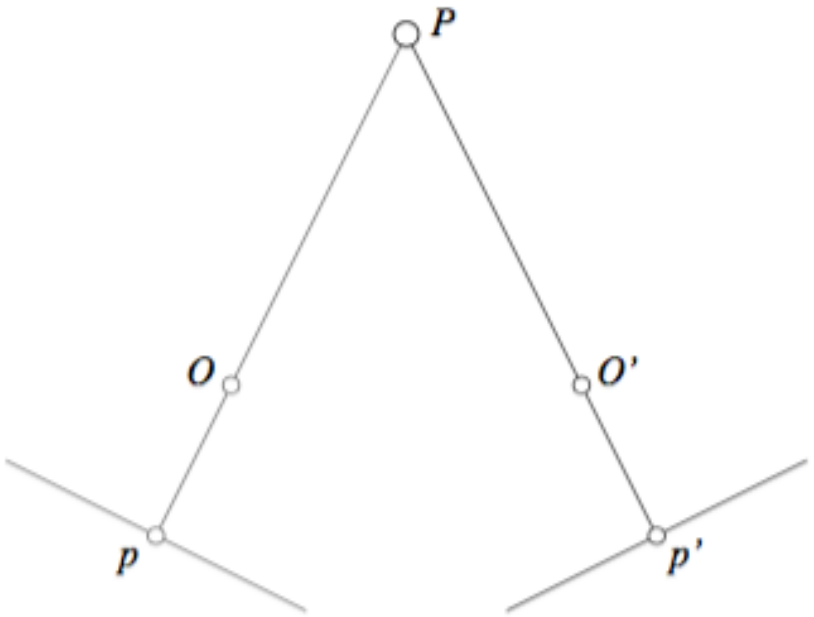
Lidar point clouds in world frame



Fused point cloud in world frame

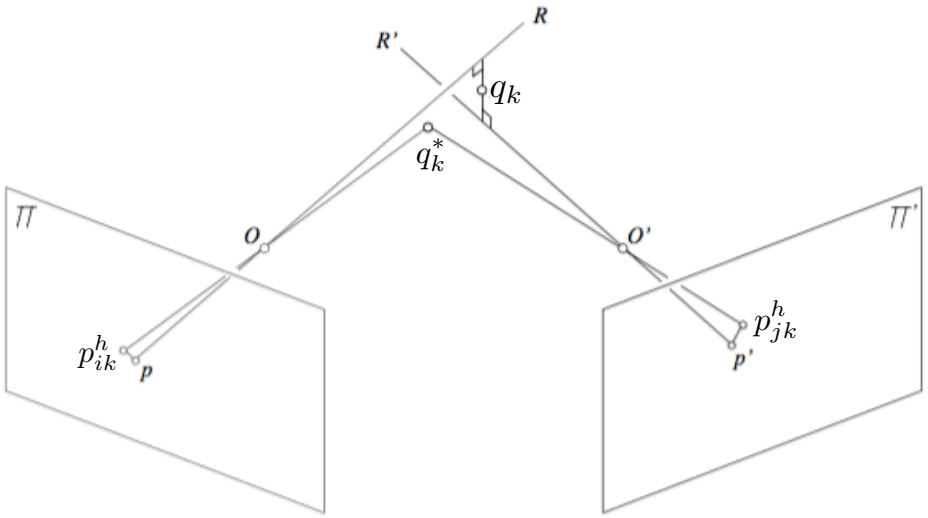
$\{P_i^* \cup P_j^*\}$

Binocular reconstruction



- Given: *calibrated* camera with known poses and two image matching points p and p'
- Find corresponding scene point by intersecting the two rays $\overline{Op}$ and $\overline{O'p'}$ (process known as triangulation)

Approximate triangulation



Camera calibration matrix

Known camera pose

RGB measurement:

$$p_{ik}^h = K R_i^T (q_k - \tau_i) + \nu_{ik}$$

$$p_{ik}^h = (u_{ik}, v_{ik}, 1) z_{ik}^h$$

Homogeneous pixel coords

Measured pixel coords in image

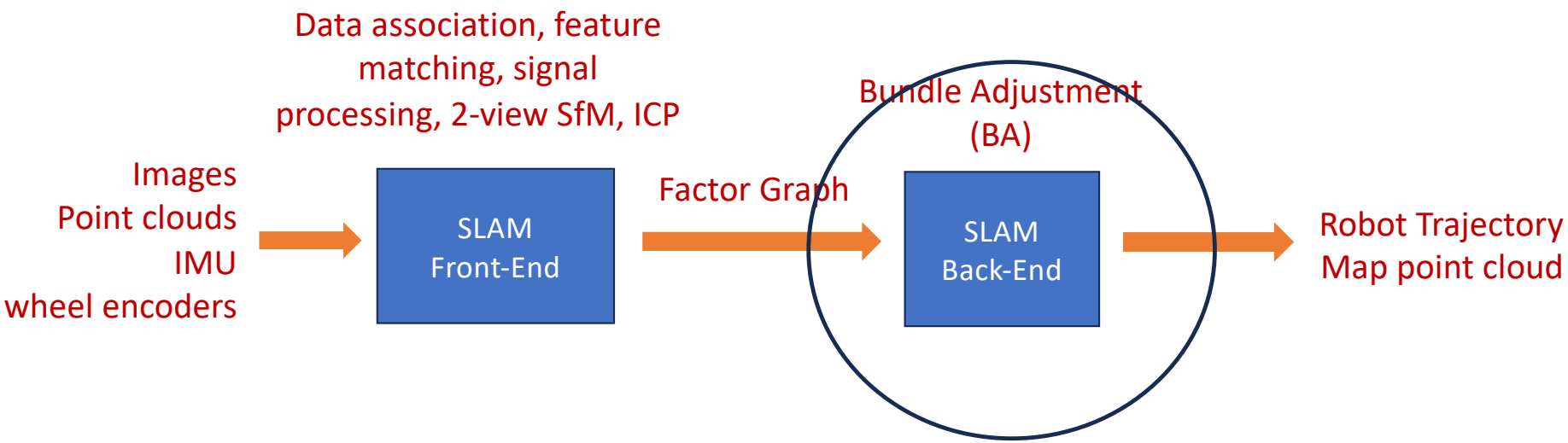
Unknown pixel depth

$$q_k^* = \arg\min_{q_k} \|R_i K^{-1} p_{ik}^h - (q_k - \tau_i)\|^2 + \|R_j K^{-1} p_{jk}^h - (q_k - \tau_j)\|^2$$

$$\text{subject to} \quad p_{ik}^h = (u_{ik}, v_{ik}, 1) z_{ik}^h$$

Factor Graphs: A Unifying Perspective for SLAM

(Frank Dellaert, 2012, 2017)



Bundle Adjustment SLAM - Produces both robot traj and map point cloud

Optimization techniques (e.g., iSAM, colmap)

Optimization Problem:

$$\min_{(R_i,\tau_i)_{i=1}^N,(q_k)_{k=1}^M} J((R_i,\tau_i)_{i=1}^N,(q_k)_{k=1}^M)$$

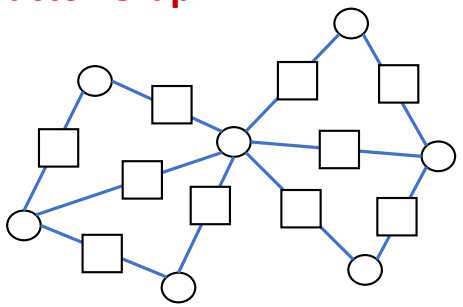
$$\text{subject to} \quad R_i^T R_i = I$$

$$\det(R_i) = 1$$

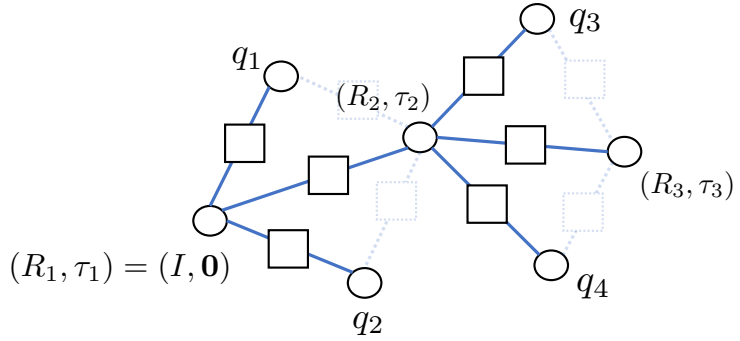
Large, non-convex, constrained problem.
In practice, many local minima.

Initialization: most optimization solvers require a rough initialization to converge to reasonable solution.

Factor Graph:



Initialization with min spanning tree:

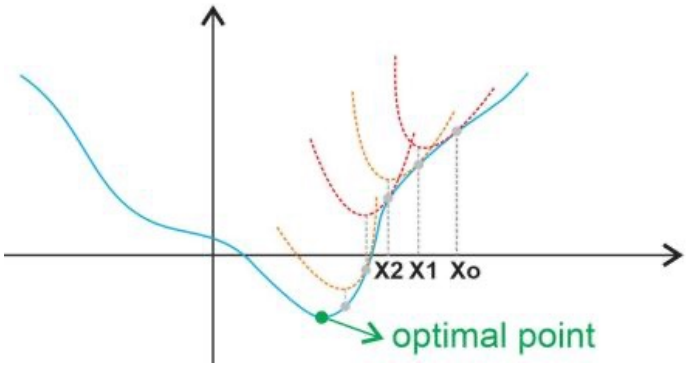


Optimization techniques (e.g., iSAM, colmap)

Optimization Problem:

$$\min_{(R_i,\tau_i)_{i=1}^N,(q_k)_{k=1}^M} J((R_i,\tau_i)_{i=1}^N,(q_k)_{k=1}^M)$$

$$\text{subject to} \quad R_i^T R_i = I$$
$$\det(R_i) = 1$$



Gauss-Newton Optimization

$$J = \sum_{(i,j)} f(T_i,T_j) = \sum_{(i,j)} \|l_{ij}(T_i,T_j)\|^2 \approx \sum_{(i,j)} \|l(T_i^0,T_j^0) + \nabla l(T_i^0,T_j^0)((T_i - T_j^0),(T_j - T_j^0))\|^2$$

Nonlinear least squares!

Linear least squares!

Taylor linearization about (T_i^0, T_j^0)

$$(T_i^1)_{i=1}^N = \arg\min \sum_{(i,j)} \|l(T_i,T_j) + \nabla l(T_i^0,T_j^0)((T_i - T_i^0),(T_j - T_j^0))\|^2$$

Iterate until convergence

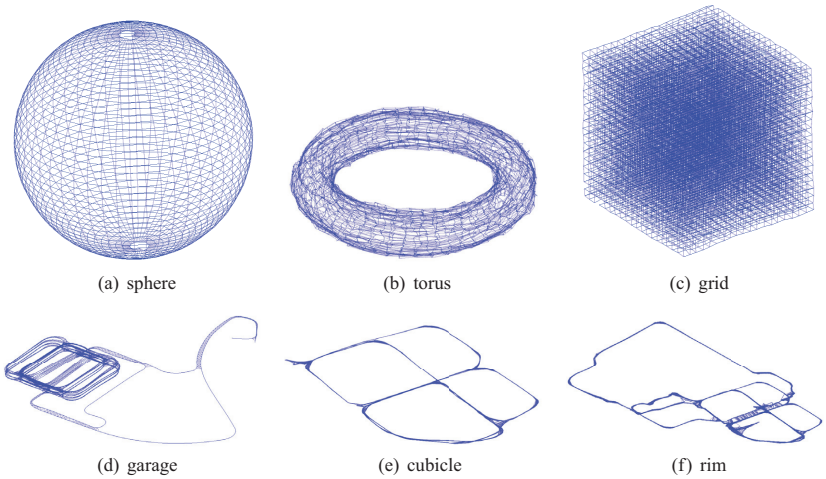
Levenberg-Marquart Optimization:

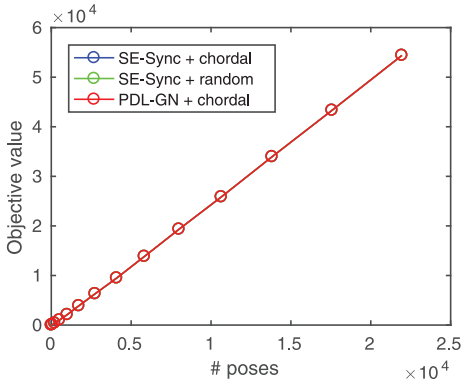
Combination of Gauss-Newton and Grad descent

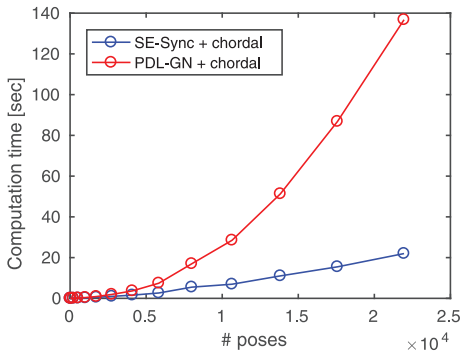
SE-Sync: Efficient Certifiable Solver for PGO

Rosen et al, IJRR 2018

Summary: Uses Riemannian optimization, duality, and semi-definite programming to solve PGO with suboptimality bound







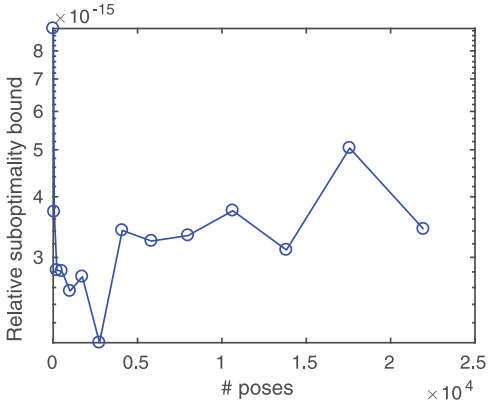
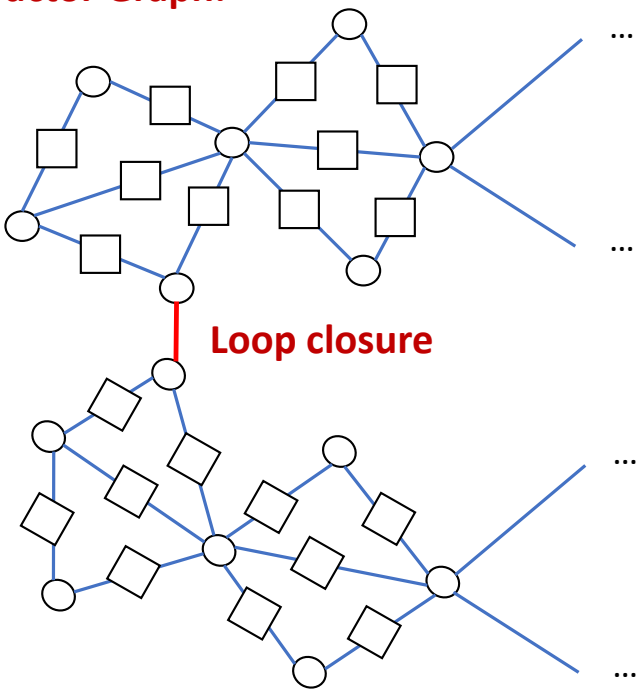
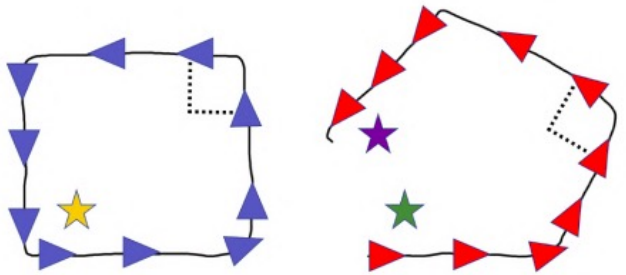


Fig. 5. Globally optimal solutions for the 3D SLAM benchmark datasets listed in Table 2.

Loop Closure

Factor Graph:





- **Long time separation**
- **Close spatial separation**
- **Requires place recognition or special loop closure module**

Loop Closure

Factor Graph:

